

Gradient Descent

- Previously, we saw visualizations of cost function J & how you can try different choices of params w & b & see what cost value they give you.

Problem:

As there are infinite values of w & b and it's not possible manually to check every value.

What would be nice?

It would be nice to have a systematic way to find values of w & b , that result in small possible cost J of w & b .

Algorithm:

Turns out there's an algorithm called "gradient descent" that you can use to do that.

→ Not just used for linear regression but for training the most advanced models (neural networks) also called "deep learning models".

we'll learn in 2nd course

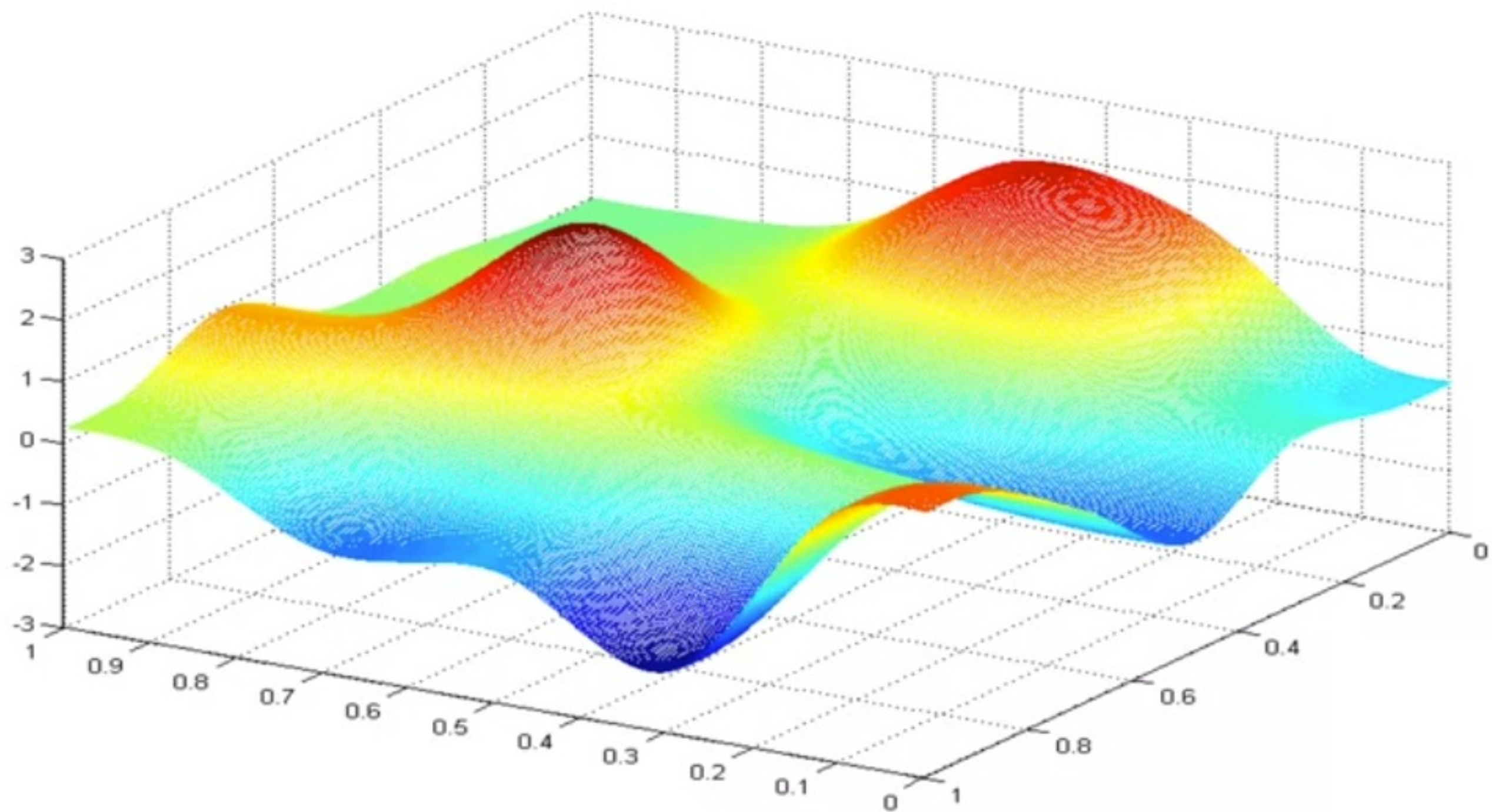
What we'll do with gradient descent:

Have some function

$$J(w, b)$$

→ c.f for linear regression, or any function

Want " $\min_{w, b} J(w, b)$ "



More general discussion:

Turns out that gradient descent applies to more general functions, including other cost functions that work with models having more than two parameters.

→ for instance:

$$J(w_1, w_2, \dots, w_n, b)$$

$$\min_{w_1, \dots, w_n, b} J(w_1, w_2, \dots, w_n, b)$$

↳ G.D. can be used to minimize this c.f. as well.

Outline:

Start with some w, b (doesn't matter what the initial values are)

→ common choice is to set them both to zero.

$$(w=0, b=0)$$

→ with G.D. ; keep changing w, b to reduce $J(w, b)$

→ until it settles at or near a minimum.

→ one thing to notice,

that for some functions J that may not be bowl shape or hammack shape, it is possible to have more than one possible minimum.

may have > 1 minimum.

As in above pic:

- $J(w, b)$ → not sq. error cost function
- for L.R with sq. error c.f., you always end up with bowl shaped or hammack shape.

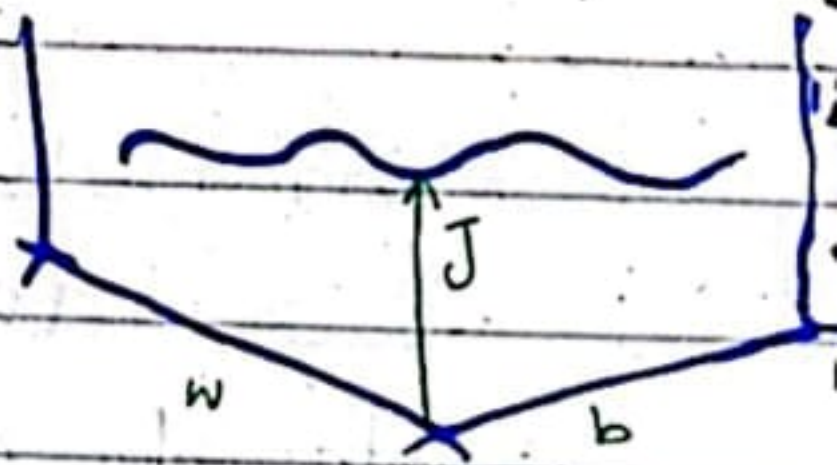
But,

this is a type of c.f. you might get if you are training a neural network model.

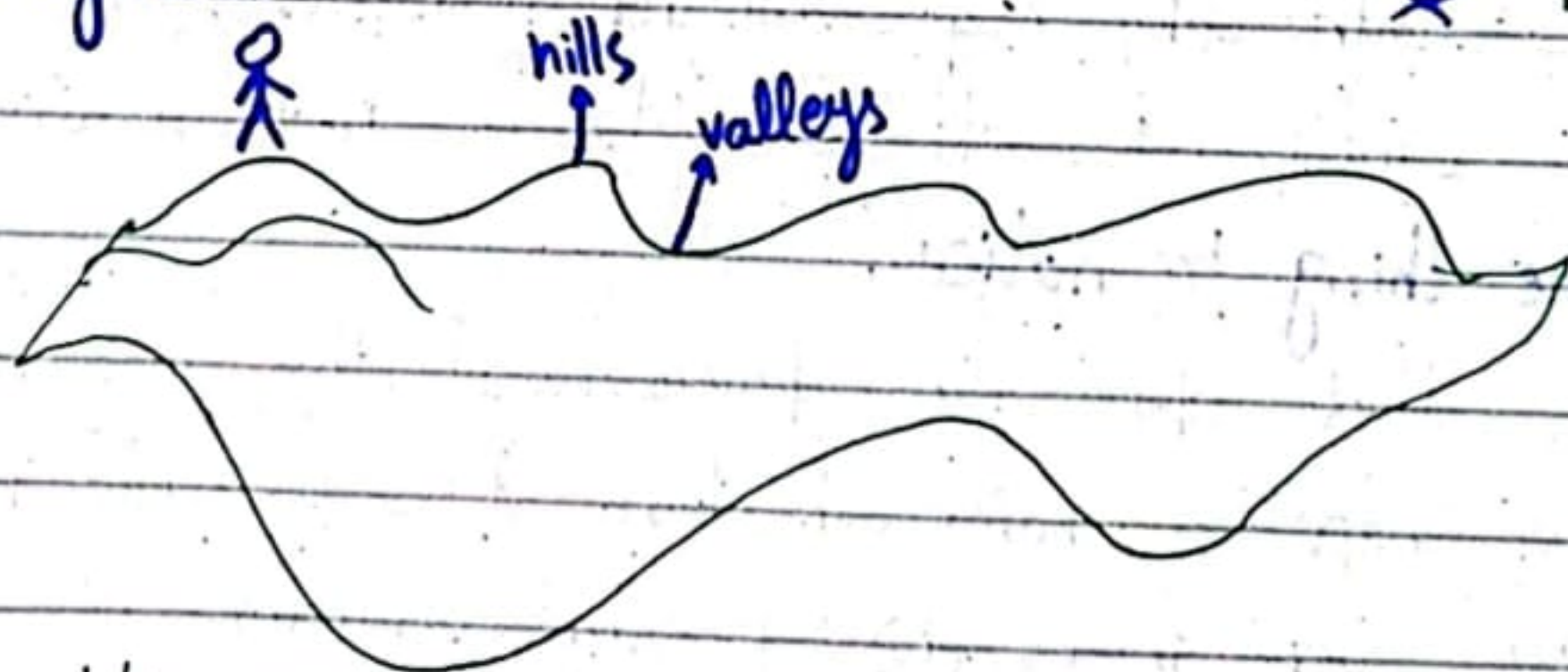
Notice,

the axis that's w & b on bottom axis

→ for diff values of w & b you get different points on this given surface.



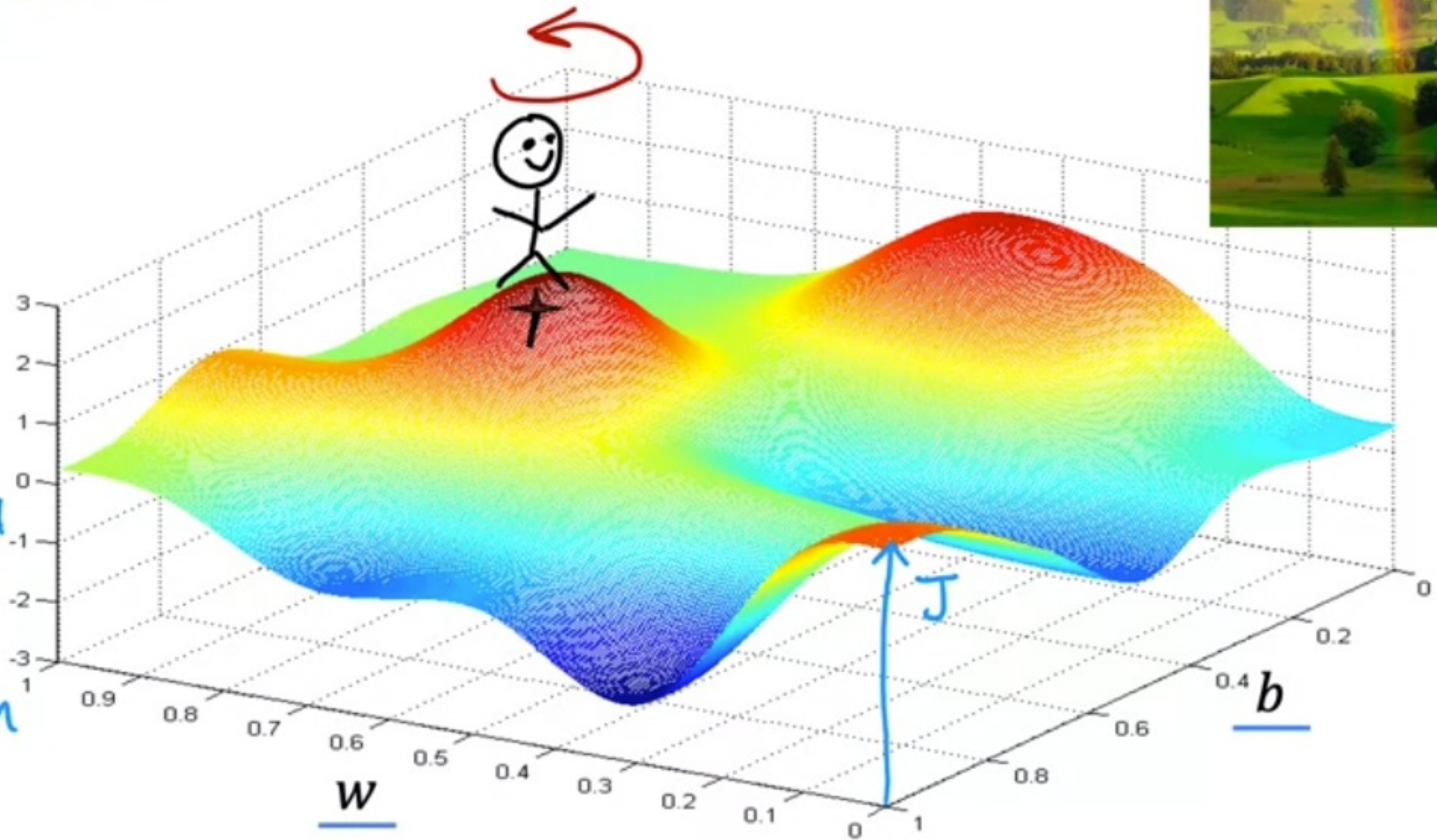
Imagine,



→ Its like golf course; your goal is to start up in hill like the person standing & get to bottom of one of these valleys as efficiently as possible.

gradient descent

$J(w, b)$
not squared
error cost
not linear
regression

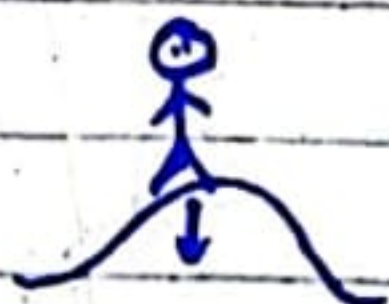


What G.D does here is,

you are going to spin 360 degrees & look around & ask your self, If I were to take tiny little baby step in one direction, and I want to go downhill as quickly as possible to go to one of these valleys.

What direction would I take that baby step in?

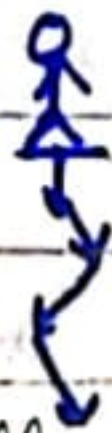
If you want to walk down as efficiently as possible. So when you look around the best direction to take your next step is roughly that direction.



Mathematically, this is the direction of steepest gradient.

means if you take this step you will go downhill faster than a step you could have taken in any other direction.

↳ now you are in new position, you again look around & check & take another step.

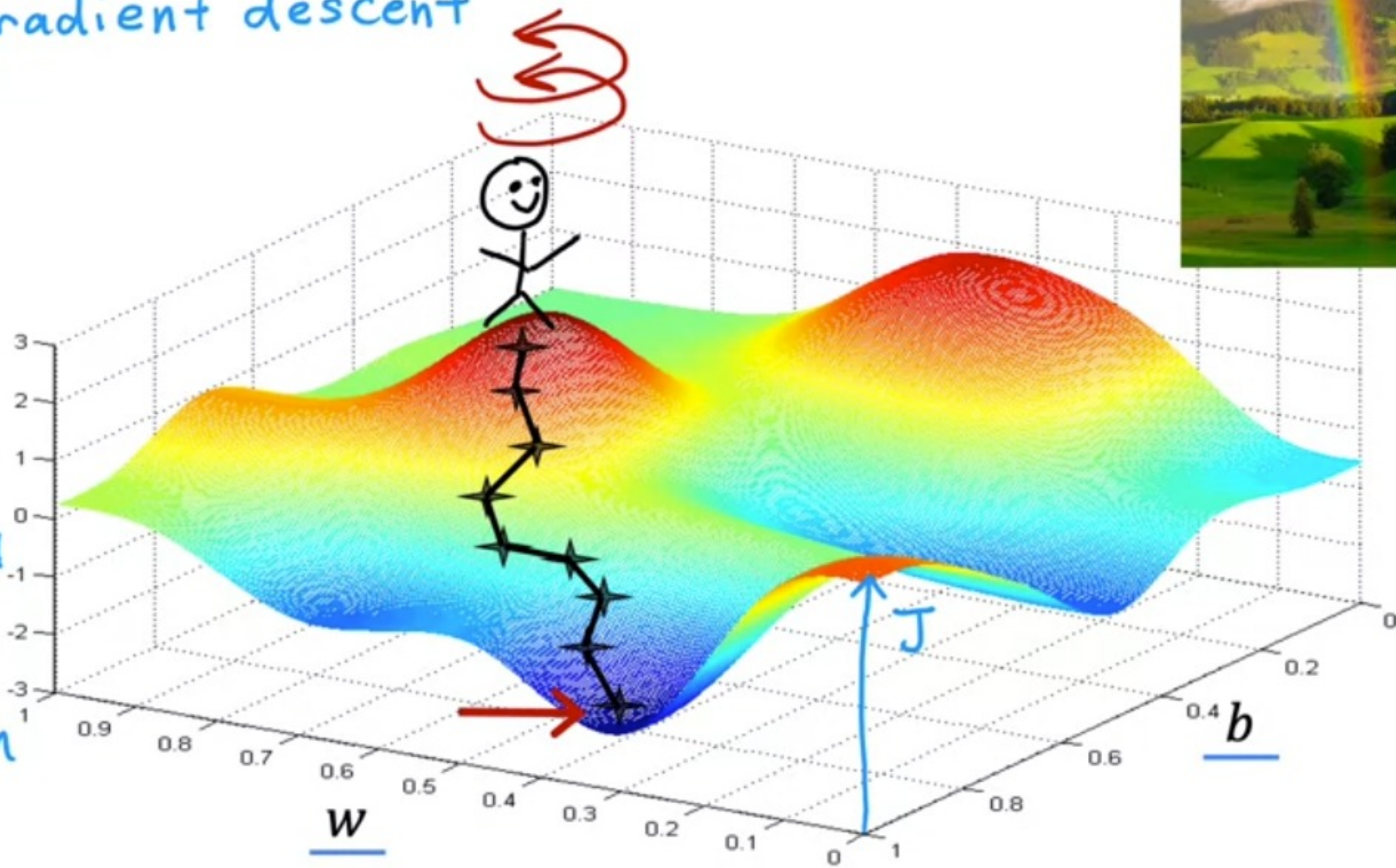


and another and so on.

until you are at bottom of valley.

gradient descent

$J(w, b)$
not squared
error cost
not linear
regression

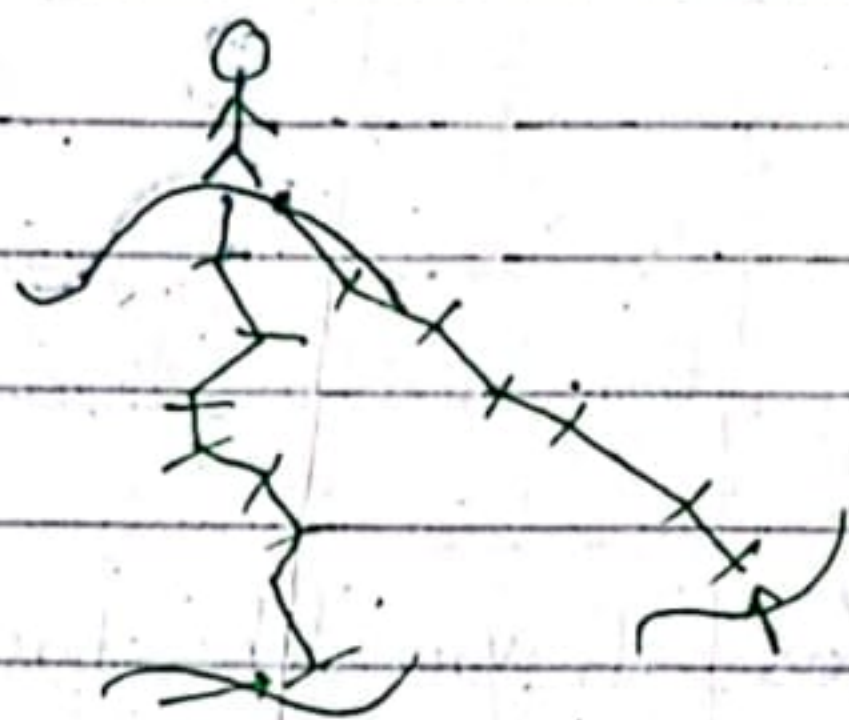


What you just did,
was go through multiple steps of
gradient descent.

Remember,

When starting G.D you choose value
of w & b . if we just like go to little
right as w, b value changed now the whole
path changes, and you end up in totally
different valley.

↳ different minimum.



The diff minimum over here on the right
→ The bottoms of both the valleys are
called "local minima".

↳ Bcz if you start going down
one valley, the g.d won't lead you to the
other one & vice versa.

→ you find your way into one ^{local} minimum &
not 2nd one.

gradient descent

$J(w, b)$
not squared
error cost
not linear
regression

